mitology vignette

2026-10-15

Abstract

mitology allows to explore the mitochondrial activity through transcriptomic profiles. It is based on a collection of genes whose proteins localize in to the mitochondria, derived from specialized databases: MitoCarta3.0 [1], IMPI [2], MSeqDR [3], and Gene Ontology [4] (genes annotated with terms related to 'mitochondri-' in their description). Then, mitology provides ready-to-use implementations of MitoCarta3.0 pathways and also a reorganization of the pathways related to mitochondria activity from Reactome [5] and Gene Ontology. These mitochondria-focused pathways can be used to conduct mitochondrial-oriented pathway analyses, enabling single-sample assessments. This vignette walks through the full process from raw gene lists to interpretable mitochondrial pathway results.

Introduction

Mitochondria are cellular organelles whose main role is related to the aerobic respiration to supply energy to the cell. In addition to this, they are also involved in other tasks, such as signaling, cellular differentiation, and cell death, as well as maintaining control of the cell cycle and cell growth. The dysfunction of mitochondria activity has been implicated in several human disorders and conditions. Studying mitochondrial behavior in health and disease conditions is opening new possibilities of understanding disease mechanisms and providing new treatments. Indeed, the possibility to target mitochondria for cancer therapies is increasingly becoming a reality in biomedical research.

The analysis of high resolution transcriptomic data can help in better understanding mitochondrial activity in relation to the gene expression dynamics. Even if a curated mitochondrial specific pathway resource exists (MitoCarta3.0, which provides a list of mitochondrial genes organized into mitochondrial pathways [1]) computational tools for mitochondrial-focused pathway analysis are still lacking. In the field of gene set analysis, general purpose resources are pathway databases, such as Reactome [5] and the Gene Ontology (GO) project [4], which aim at providing gene set categories for high-throughput data analysis for all the cellular contexts. In the vast majority of pathways contained in these public databases, the mitochondrial-specific component of cellular signaling constitutes only a small portion, which is often overshadowed by the non-mitochondrial signaling. And, even if the information on protein localization is present, pathway analysis typically does not account for whether the activated signal involves the mitochondrion or not.

Thus, we thought that a tool able to focus specifically on the mitochondrial part of pathways could allow tailored and more specific analysis on this interesting organelle and we started to develop mitology to provide actionable and fast mitochondrial phenodata interpretation of high-throughput data.

These vignete shows some ways to perform mitochondrial pathway analyses using mitology. Also, some visualization functions are implemented to visualize the scores. These can help in the result interpretations.

Installation

To install this package:

```
if (!require("BiocManager", quietly = TRUE))
    install.packages("BiocManager")

BiocManager::install("mitology")
```

Also install the other packages used in this vignette, if you do not have them already:

```
BiocManager::install(c("org.Hs.eg.db", "AnnotationDbi", "GSVA"))
```

Mitochondrial Genes

A list of mitochondrial genes was obtained by joining four public resources that were developed in the last years to define the mitochondrial proteome with the aim to study mitochondrial functions, dysfunctions and diseases. Specifically, we collected the genes from: MitoCarta3.0 [1], the Integrated Mitochondrial Protein Index (IMPI) [2], the Mitochondrial Disease Sequence Data Resource (MSeqDR) [3] and The Gene Ontology (GO) [4].

In the MitoGenesDB table we can see the full list of genes along with the database from which it was retrieved.

```
library(mitology)
```

```
##
```

```
##
```

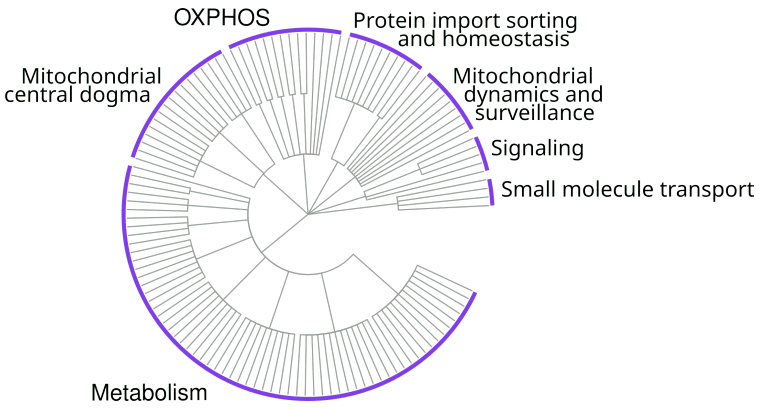
```
data(MitoGenesDB)
head(MitoGenesDB)
```

```
##           ENSEMBL SYMBOL        DB
## 1 ENSG00000000003 TSPAN6      IMPI
## 2 ENSG00000000419   DPM1      MSDR
## 3 ENSG00000000938    FGR      GOCC
## 4 ENSG00000001084   GCLC IMPI,GOCC
## 5 ENSG00000001460  STPG1      GOCC
## 6 ENSG00000001626   CFTR      IMPI
```

Gene sets collected in mitology

Gene sets from MitoCarta3.0

The whole and original structure from the MitoCarta3.0 database was kept and included in mitology organized in a three-tier hierarchy.



In addition to MitoCarta3.0, we decided to propose three more comprehensive mitochondrial-oriented gene set resources exploiting Reactome, GO Biological Processes (GO-BP), and GO Cellular Components (GO-CC).

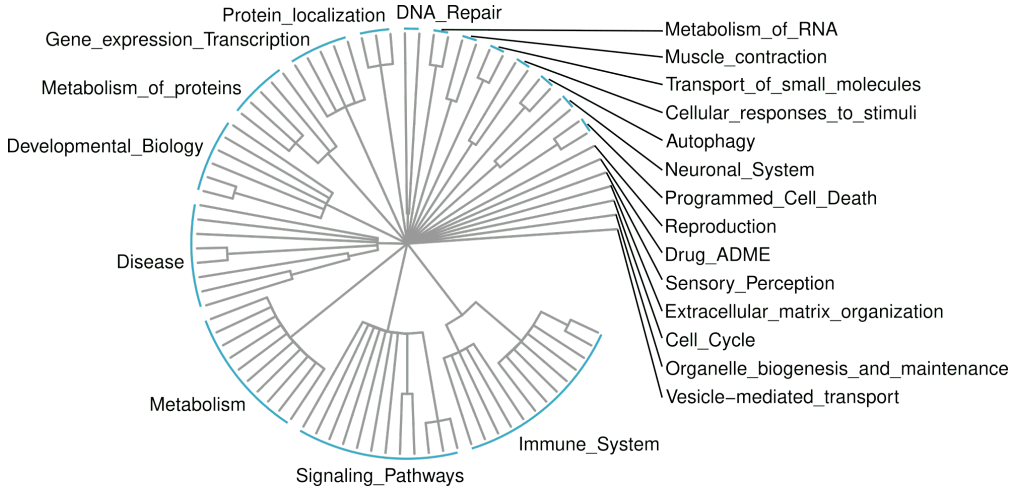
Gene sets from GO

We explored the GO-CC and GO-BP terms. To filter the GO terms we performed a four-steps selection. Firstly, terms were filtered by size, keeping all the ones with at least 10 mitochondrial genes. Then, we pruned the GO trees of CC and BP by the graph levels, filtering out the 0°, 1°, 2° and 3° levels and keeping the terms from the 4° level. Then, we tested the enrichment of the remaining sets for our mitochondrial gene list with the `enrichGO` function from the `clusterProfiler` R package (v4.14.3). Only sets with FDR lower than 0.05 have been passed to the next step. Finally, the last selection was topological, we exploited the GO hierarchical organization selecting the more general enriched gene sets filtering out their offspring terms.



Gene sets from Reactome

The Reactome pathways were retrieved with the `graphite` R package (v1.52.0). Reactome has a simpler structure with far less level of nested pathways, thus we applied only three filtering steps: number of mitochondrial genes over 10; FDR under 0.05 (enrichment computed with the `phyper` function) and filtering of the offspring pathways.



Once we obtained the final terms and pathways respectively from GO and Reactome, we extracted the mitochondrial gene sets by keeping only the genes included in the mitochondrial list. The same names of the original terms/pathways were kept for the gene sets included in mitology. The final gene sets and the corresponding tree-structures of the four databases were kept and included in the mitology package.

Access mitochondrial gene sets

The `getGeneSets` function allows to get the mitochondrial gene sets. It returns them for one of the four possible databases (MitoCarta, Reactome, GO-CC and GO-BP) by the `database` argument. The `nametype` ar-

gument says the type of gene name ID, either one of SYMBOL, ENTREZID or ENSEMBL. The `objectType` argument can be set to return the gene sets in form of a list or a data frame.

```
MC_df <- getGeneSets(
    database = "MitoCarta", nametype = "SYMBOL", objectType = "dataframe")

MC_list <- getGeneSets(
    database = "MitoCarta", nametype = "SYMBOL", objectType = "list")
```

How to use mitology

Questions you need to answer first

Before starting, answer these questions about your data.

Question 1 — What kind of gene list do you have?

Your situation	What you need	mitology function
A list of genes that are significantly different between two conditions	**A vector of gene IDs**	`enrichMito()` (over-representation analysis, ORA)
A ranked list covering all tested genes, e.g. ordered by fold-change or a test statistic	**A named, sorted numeric vector**	`gseaMito()` (gene set enrichment analysis, GSEA)
Expression data for many samples and you want one mitochondrial-activity score per sample (e.g., to compare groups, cluster samples, or draw a heatmap)	**An expression matrix**	`getGeneSets()` + `GSVA::gsva()` (e.g. ssGSEA or GSVA)

Question 2 — Which database do you want to use?

`mitology` offers four options. If you are unsure, start with `"MitoCarta"` — it is the best default for a first analysis.

Database	Best used when
`"MitoCarta"`	You want the most experimentally validated set of mitochondrial pathways.
`"Reactome"`	You want broader biological context (mitochondria-related signaling, disease pathways, metabolism) beyond the curated MitoCarta pathways.
`"GO-BP"`	You are interested in mitochondria-related biological processes (e.g. specific metabolic or signaling activities).
`"GO-CC"`	You are interested in mitochondria-related sub-structures and localization (e.g. specific complexes or membrane compartments).

Get your gene IDs into ENSEMBL format

Why this step matters: `mitology`'s gene sets are indexed by ENSEMBL gene ID. If your data uses gene symbols (e.g. `"TP53"`, `"SOD2"`) or another ID type, you must convert them first, or `mitology` will not recognize your genes and will report zero results.

If your genes are already ENSEMBL IDs (e.g. `"ENSG00000141510"`), skip this step.

Otherwise, convert them like this:

```
library(org.Hs.eg.db)
library(AnnotationDbi)

# Replace this with your own gene symbols
my_gene_symbols <- c("SOD2", "TP53", "PPARGC1A", "TFAM")

my_gene_ensembl <- mapIds(
    org.Hs.eg.db,
    keys = my_gene_symbols,
    column = "ENSEMBL",
    keytype = "SYMBOL",
    multiVals = "first"
)

my_gene_ensembl <- my_gene_ensembl[!is.na(my_gene_ensembl)]
my_gene_ensembl
```

```
##              SOD2              TP53          PPARGC1A              TFAM
## "ENSG00000291237" "ENSG00000141510" "ENSG00000109819" "ENSG00000108064"
```

Enrichment analyses

In the following section, we use an example dataset included in `mitology` (`ovse`, bulk gene expression from 40 ovarian cancer samples) to show how to perform an analysis of the mitochondrial activity.

```
# loading packages
library(SummarizedExperiment)
library(Biobase)
```

```
# load data
data(ovse)
ovse
```

```
## class: SummarizedExperiment
## dim: 2388 40
## metadata(0):
## assays(1): norm_expr
## rownames(2388): ABCA4 ABCB10 ... USP9X WDR45
## rowData names(2): PROvsIMR_logFC PROvsIMR_FDR
## colnames(40): sample1 sample2 ... sample39 sample40
## colData names(1): OV_subtype
```

Enrichment analyses of the mitochondrial gene sets

We can perform an over representation analysis (ORA) with `enrichMito()`.

```
# --- Using the example data: genes significant between two OV subtypes ---
genes <- rownames(ovse)[elementMetadata(ovse)$PROvsIMR_FDR < 0.01]
```

```r
genes <- mapIds(
    org.Hs.eg.db, keys = genes, column = "ENSEMBL",
    keytype = "SYMBOL", multiVals = "first")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```r
genes <- genes[!is.na(genes)]

# --- Replace the vector with your list of genes ---
# genes <- my_gene_ensembl

enrichresMC <- enrichMito(genes = genes, database = "MitoCarta")
enrichresRT <- enrichMito(genes = genes, database = "Reactome")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```r
head(as.data.frame(enrichresMC))
```

```
##                                                                    MitoCarta.ID
## ROS_and_glutathione_metabolism             ROS_and_glutathione_metabolism
## Fusion                                                                 Fusion
## Fe-S-containing_proteins                       Fe-S-containing_proteins
## Mitochondrial_ribosome                           Mitochondrial_ribosome
## Proteases                                                     Proteases
## Branched-chain_amino_acid_metabolism Branched-chain_amino_acid_metabolism
##                                                             MitoCarta.Description
## ROS_and_glutathione_metabolism             ROS_and_glutathione_metabolism
## Fusion                                                                 Fusion
## Fe-S-containing_proteins                       Fe-S-containing_proteins
## Mitochondrial_ribosome                           Mitochondrial_ribosome
## Proteases                                                     Proteases
## Branched-chain_amino_acid_metabolism Branched-chain_amino_acid_metabolism
##                                      MitoCarta.GeneRatio MitoCarta.BgRatio
## ROS_and_glutathione_metabolism                     9/164           28/1049
## Fusion                                             4/164           10/1049
## Fe-S-containing_proteins                           9/164           33/1049
## Mitochondrial_ribosome                            20/164           91/1049
## Proteases                                          8/164           30/1049
## Branched-chain_amino_acid_metabolism               7/164           27/1049
##                                      MitoCarta.RichFactor
## ROS_and_glutathione_metabolism                  0.3214286
## Fusion                                          0.4000000
## Fe-S-containing_proteins                        0.2727273
## Mitochondrial_ribosome                          0.2197802
## Proteases                                       0.2666667
## Branched-chain_amino_acid_metabolism            0.2592593
##                                      MitoCarta.FoldEnrichment
## ROS_and_glutathione_metabolism                       2.055967
## Fusion                                               2.558537
## Fe-S-containing_proteins                             1.744457
## Mitochondrial_ribosome                               1.405789
## Proteases                                            1.705691
## Branched-chain_amino_acid_metabolism                 1.658311
##                                      MitoCarta.oddsRatio MitoCarta.zScore
## ROS_and_glutathione_metabolism                  2.646520         2.436954
```

```
## Fusion                                          3.662500         2.130788
## Fe-S-containing_proteins                        2.083065         1.869737
## Mitochondrial_ribosome                          1.592332         1.742888
## Proteases                                       2.011655         1.687404
## Branched-chain_amino_acid_metabolism            1.928344         1.491140
##                                     MitoCarta.pvalue MitoCarta.p.adjust
## ROS_and_glutathione_metabolism           0.02121101          0.7619992
## Fusion                                   0.05614461          0.7619992
## Fe-S-containing_proteins                 0.05879251          0.7619992
## Mitochondrial_ribosome                   0.05976464          0.7619992
## Proteases                                0.08190672          0.8354485
## Branched-chain_amino_acid_metabolism     0.11403898          0.9693313
##                                     MitoCarta.qvalue
## ROS_and_glutathione_metabolism             0.7619992
## Fusion                                     0.7619992
## Fe-S-containing_proteins                   0.7619992
## Mitochondrial_ribosome                     0.7619992
## Proteases                                  0.8354485
## Branched-chain_amino_acid_metabolism       0.9693313
##
## ROS_and_glutathione_metabolism
## Fusion
## Fe-S-containing_proteins
## Mitochondrial_ribosome              ENSG00000055950/ENSG00000182180/ENSG00000143314/ENSG00000158042/
## Proteases
## Branched-chain_amino_acid_metabolism
##                                     MitoCarta.Count
## ROS_and_glutathione_metabolism                    9
## Fusion                                            4
## Fe-S-containing_proteins                          9
## Mitochondrial_ribosome                           20
## Proteases                                         8
## Branched-chain_amino_acid_metabolism              7
```

Each row is a mitochondrial gene set. The columns you will look at most are pvalue/p.adjust (statistical significance — lower is more significant) and Count (how many of your genes fall in that gene set).

Gene set enrichment analysis (GSEA)

With the full ranked gene list we can also compute the GSEA with `gseaMito()`.

```
# --- Using the example data: all genes ranked by fold-change ---
geneslFC <- elementMetadata(ovse)$PROvsIMR_logFC
names(geneslFC) <- rownames(ovse)
names(geneslFC) <- mapIds(
    org.Hs.eg.db, keys = names(geneslFC), column = "ENSEMBL",
    keytype = "SYMBOL", multiVals = "first")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```
geneslFC <- geneslFC[!is.na(names(geneslFC))]
geneslFC <- sort(geneslFC, decreasing = TRUE)

# --- Replace the block above with your own named, sorted numeric vector: ---
# names(geneslFC) must be ENSEMBL IDs, values must be your ranking statistic
# (e.g. log fold-change or a t-statistic), sorted from highest to lowest.
```

```
gsearesMC <- gseaMito(genes = geneslFC, database = "MitoCarta")
gsearesRT <- gseaMito(genes = geneslFC, database = "Reactome")
```

`NES` (normalized enrichment score) tells you the direction and strength of the signal: positive means the gene set is shifted toward the top of your ranked list, negative means it is shifted toward the bottom.

Per-sample score on many samples

You may want to compute single-sample GSEA (ssGSEA) with `GSVA` package.

```
library(GSVA)
MC_list <- getGeneSets(database = "MitoCarta", nametype = "SYMBOL", objectType = "list")

# --- Replace `ovse` below with your own expression matrix or
#     SummarizedExperiment object ---
gsvaPar <- ssgseaParam(exprData = ovse, geneSets = MC_list)
```

```
## i No assay name provided; using default assay 'norm_expr'
```

```
res_ssGSEA <- gsva(gsvaPar)
```

```
## i GSVA version 2.6.12
## i Searching for rows with constant values
## ! 5 rows with constant values throughout the columns
## ! Some gene sets have size one. Consider setting minSize > 1
## i Calculating ssGSEA scores for 119 gene sets
## i Normalizing ssGSEA scores
## v Calculations finished
```

Important: `gsva()` returns a `SummarizedExperiment`-like object, not a plain matrix — it packages the scores together with sample/gene set metadata. The visualization functions (`mitoHeatmap()`, `mitoTreeHeatmap()`) expect a plain matrix, so extract it with `assay()` before moving on:

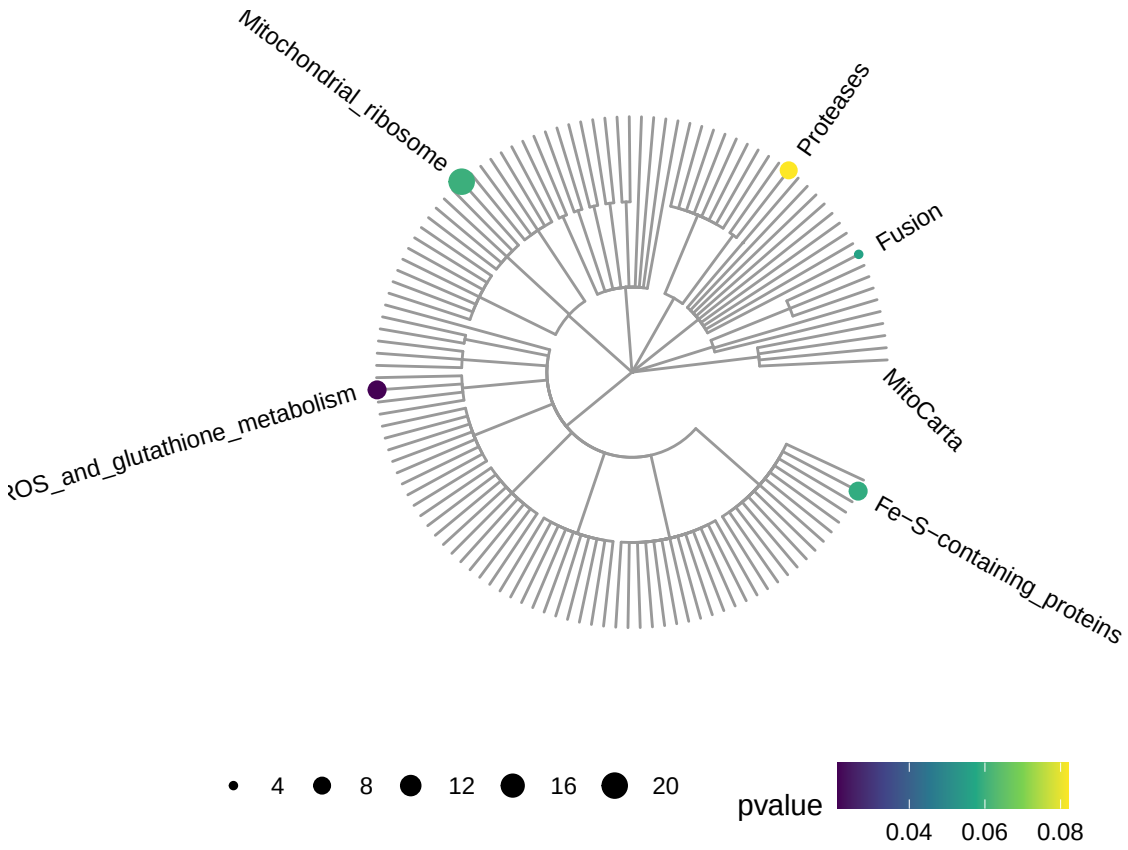
```
res_ssGSEA_matrix <- assay(res_ssGSEA)
```

`res_ssGSEA_matrix` now contains one mitochondrial-activity score per gene set (rows) per sample (columns) — this is the input you need for the heatmaps in the visualization step.

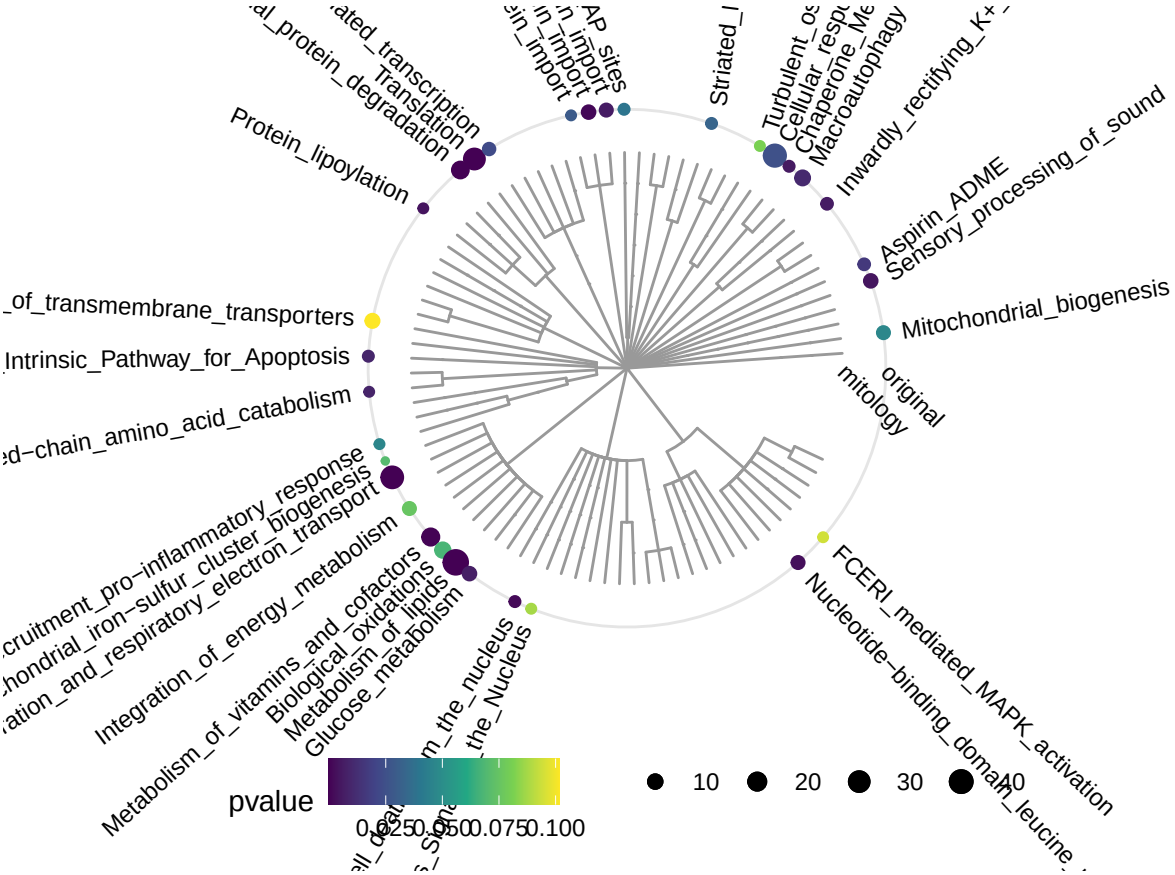
Visualize the results

The results obtained with the ORA and GSEA can be visualized with a dot plot over the database tree hierarchy.

```
mitoTreePoint(data = enrichresMC, database = "MitoCarta",
              pvalCutoff = .9, color = "pvalue")
```




```
mitoTreePoint(data = enrichresRT, database = "Reactome",
              pvalCutoff = .6, color = "pvalue")
```



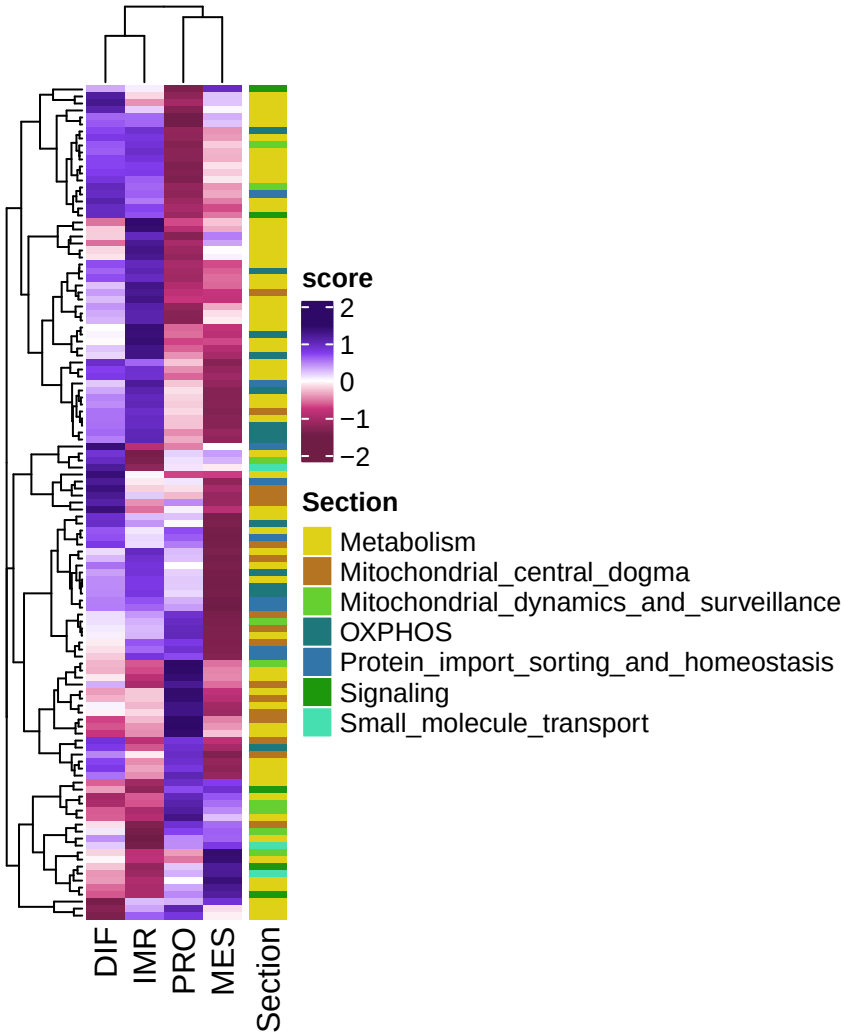
Each dot is a mitochondrial gene set; dot size and color encode significance (see the `color` argument to switch between `"pvalue"`, `"p.adjust"`, or `"NES"` for GSEA results). Lower `pvalCutoff` values show fewer, more stringent results.

Instead, when we obtain a matrix of scores for each sample in each gene set we can visualize it in an heatmap.

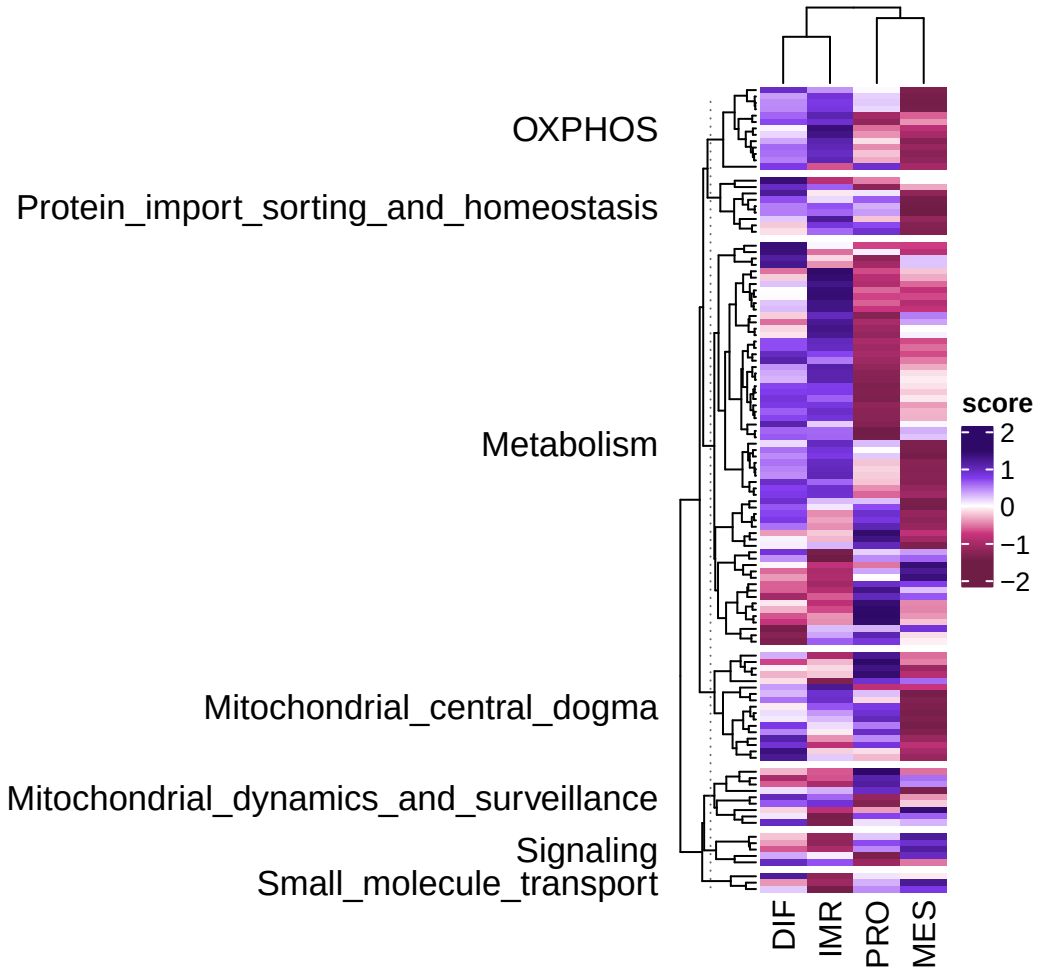
Since we obtained a single score for each sample, it would be difficult to visualize all of them. If your samples belong to a few known groups/conditions (as in our example, where 40 samples belong to 4 ovarian cancer subtypes), average the scores within each group before plotting:

```
res_ssGSEA_subtype <- do.call(
    cbind, lapply(unique(ovse$OV_subtype), function(x){
        rowMeans(res_ssGSEA_matrix[, ovse$OV_subtype==x])
    }))
colnames(res_ssGSEA_subtype) <- unique(ovse$OV_subtype)
rownames(res_ssGSEA_subtype) <- rownames(res_ssGSEA_matrix)
res_ssGSEA_subtype <- t(scale(t(res_ssGSEA_subtype)))
```

```
set.seed(789)
mitoHeatmap(data = res_ssGSEA_subtype, database = "MitoCarta")
```

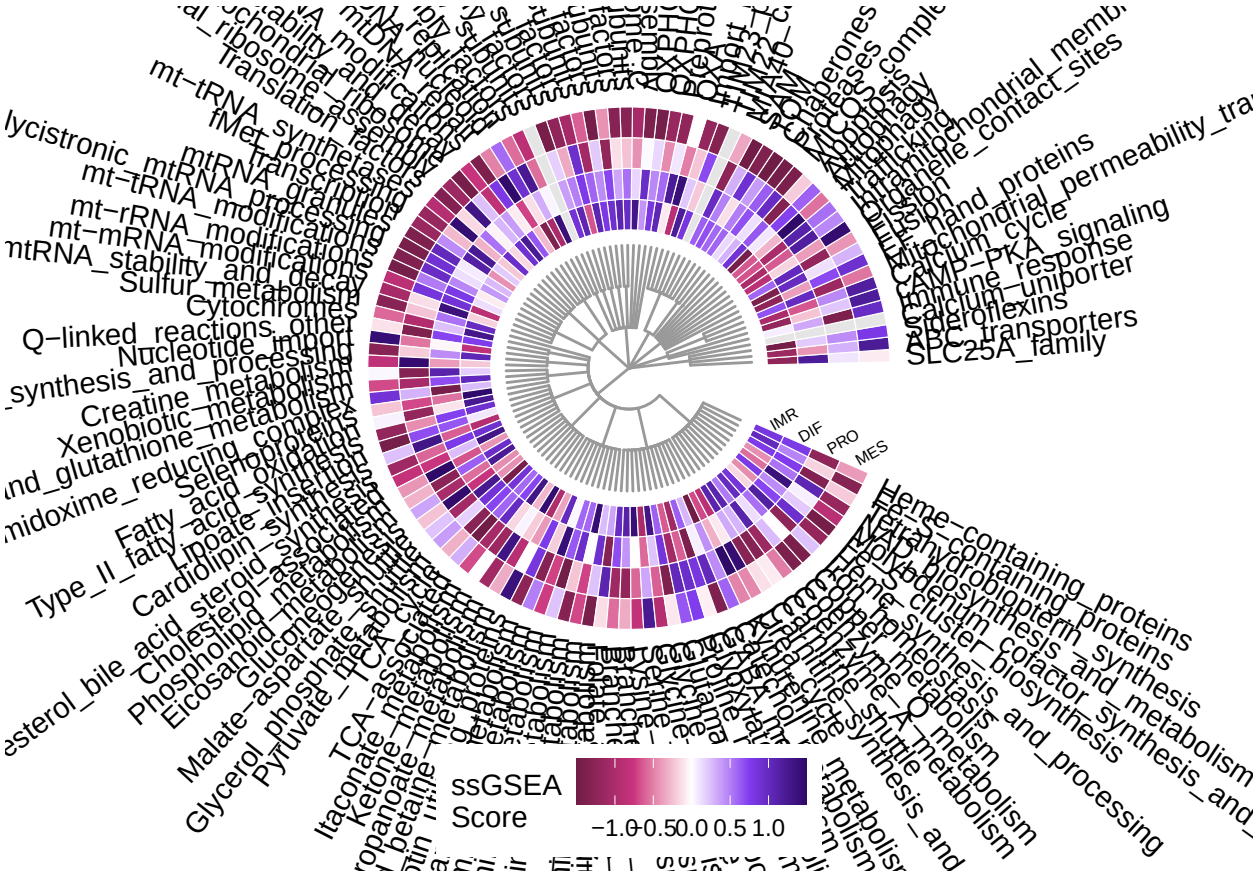



```
mitoHeatmap(data = res_ssGSEA_subtype, database = "MitoCarta", splitSections = TRUE)
```



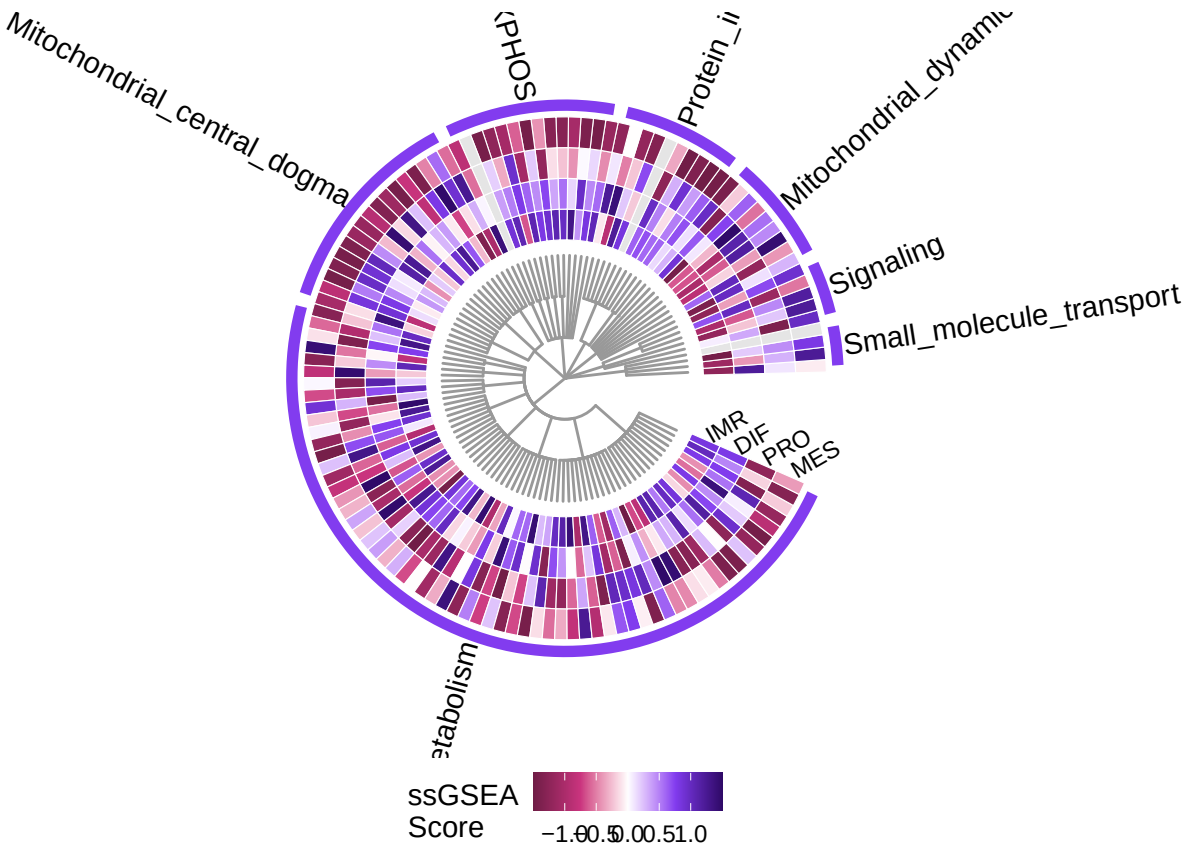
We can also visualize it by plotting a circular heatmap on the gene set tree hierarchy.

```
mitoTreeHeatmap(
    data = res_ssGSEA_subtype, database = "MitoCarta",
    labelNames = "leaves", font.size = 2)
```



It is possible to plot only the main section arguments instead of the single gene set names.

```
mitoTreeHeatmap(
    data = res_ssGSEA_subtype, database = "MitoCarta",
    labelNames = "sections", font.size = 3)
```



If you don't have groups, or want to keep per-sample resolution, you can instead simply hide the sample labels rather than aggregating:

```
mitoHeatmap(data = res_ssGSEA_matrix, database = "MitoCarta")
```

Once you have a working analysis, repeat with database = "Reactome", "GO-BP", or "GO-CC" to see whether broader biological context changes your interpretation. No other code needs to change.

Troubleshooting / FAQ

My `enrichMito`/`gseaMito` result has zero significant gene sets. This is a valid result, not necessarily an error — it means your gene list does not show statistically concentrated mitochondrial signal for the chosen database. First, double-check that your gene IDs were successfully converted to ENSEMBL and that `genes` is not empty or full of `NA`. Then, try a less stringent significance threshold on your input gene list, or try a different `database` option.

How many genes do I need for a meaningful ORA analysis? There is no hard minimum, but very short gene lists (under ~20-30 genes) rarely produce statistically significant results, since ORA depends on the number of overlapping genes. If your list is short, `gseaMito()`, which uses the full ranked list rather than a fixed cutoff, may be more appropriate.

Can I use mitology with non-human data? No — mitology's gene sets are currently built for human data only.

Where do I get help if I'm stuck? Open an issue on the mitology GitHub repository: https://github.com/CaluraLab/mitology/issues. Please include the exact code you ran, the error message (if any), and the output of `sessionInfo()`.

Bibliography

[1] Rath S, Sharma R, Gupta R, et al. MitoCarta3.0: an updated mitochondrial proteome now with sub-organelle localization and pathway annotations. Nucleic Acids Res 2020; 49:D1541–D1547

[2] Peck P. IMPI. 2021

[3] Shen L, Diroma MA, Gonzalez M, et al. MSeqDR: A Centralized Knowledge Repository and Bioinformatics Web Resource to Facilitate Genomic Investigations in Mitochondrial Disease. Hum Mutat 2016; 37:540–548

[4] The Gene Ontology Resource: 20 years and still GOing strong. Nucleic Acids Res 2019; 47:D330–D338

[5] Rothfels K, Milacic M, Matthews L, et al. Using the Reactome Database. Curr Protoc 2023; 3:e722

Session Info

Here is the output of sessionInfo() on the system on which this document was compiled.

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=it_IT.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=it_IT.UTF-8        LC_COLLATE=it_IT.UTF-8
##  [5] LC_MONETARY=it_IT.UTF-8    LC_MESSAGES=it_IT.UTF-8
##  [7] LC_PAPER=it_IT.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=it_IT.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Europe/Rome
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats4    stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] GSVA_2.6.12                SummarizedExperiment_1.42.0
##  [3] GenomicRanges_1.64.0       Seqinfo_1.2.0
##  [5] MatrixGenerics_1.24.0      matrixStats_1.5.0
##  [7] org.Hs.eg.db_3.23.1        AnnotationDbi_1.74.0
##  [9] IRanges_2.46.0             S4Vectors_0.50.2
## [11] Biobase_2.72.0             BiocGenerics_0.58.1
## [13] generics_0.1.4             mitology_1.4.0
##
## loaded via a namespace (and not attached):
##   [1] RColorBrewer_1.1-3          rstudioapi_0.19.0
##   [3] jsonlite_2.0.0              shape_1.4.6.1
```

```
##   [5] tidydr_0.0.7                magrittr_2.0.5
##   [7] magick_2.9.1                ggtangle_0.1.4
##   [9] farver_2.1.2                rmarkdown_2.32
##  [11] GlobalOptions_0.1.4         fs_2.1.0
##  [13] vctrs_0.7.3                 Cairo_1.7-0
##  [15] DelayedMatrixStats_1.34.0   memoise_2.0.1
##  [17] ggtree_4.2.0                S4Arrays_1.12.1
##  [19] htmltools_0.5.9             Rhdf5lib_2.0.0
##  [21] rhdf5_2.56.1                SparseArray_1.12.3
##  [23] gridGraphics_0.5-1          htmlwidgets_1.6.4
##  [25] httr2_1.3.0                 plyr_1.8.9
##  [27] cachem_1.1.0                igraph_2.3.4
##  [29] lifecycle_1.0.5             iterators_1.0.14
##  [31] pkgconfig_2.0.3             Matrix_1.7-6
##  [33] R6_2.6.1                    fastmap_1.2.0
##  [35] gson_0.2.2                  clue_0.3-68
##  [37] digest_0.6.39               aplot_0.3.2
##  [39] enrichplot_1.32.1           colorspace_2.1-4
##  [41] ggnewscale_0.5.2            patchwork_1.3.2
##  [43] aisdk_1.4.12                ps_1.9.3
##  [45] RSQLite_3.53.3              labeling_0.4.3
##  [47] abind_1.4-8                 httr_1.4.9
##  [49] polyclip_1.10-7             compiler_4.6.1
##  [51] bit64_4.8.6                 fontquiver_0.2.1
##  [53] withr_3.0.3                 doParallel_1.0.17
##  [55] S7_0.2.2                    graphite_1.58.0
##  [57] BiocParallel_1.46.0         viridis_0.6.5
##  [59] DBI_1.3.0                   HDF5Array_1.40.0
##  [61] ggforce_0.5.0               MASS_7.3-66
##  [63] DelayedArray_0.38.2         rappdirs_0.3.4
##  [65] rjson_0.2.23                tools_4.6.1
##  [67] otel_0.2.0                  ape_5.8-1
##  [69] scatterpie_0.2.6            glue_1.8.1
##  [71] h5mread_1.4.1               callr_3.8.0
##  [73] rhdf5filters_1.24.1         nlme_3.1-171
##  [75] GOSemSim_2.38.3             grid_4.6.1
##  [77] cluster_2.1.8.3             reshape2_1.4.5
##  [79] memuse_4.2-3                gtable_0.3.6
##  [81] tidyr_1.3.2                 tidygraph_1.3.1
##  [83] XVector_0.52.0              ggrepel_0.9.8
##  [85] foreach_1.5.2               pillar_1.11.1
##  [87] stringr_1.6.0               yulab.utils_0.2.5
##  [89] splines_4.6.1               circlize_0.4.18
##  [91] dplyr_1.2.1                 tweenr_2.0.3
##  [93] treeio_1.36.1               lattice_0.23-1
##  [95] bit_4.6.0                   annotate_1.90.0
##  [97] tidyselect_1.2.1            SingleCellExperiment_1.34.0
##  [99] fontLiberation_0.1.0        GO.db_3.23.1
## [101] ComplexHeatmap_2.28.0       Biostrings_2.80.2
## [103] knitr_1.52                  reactome.db_1.96.0
## [105] fontBitstreamVera_0.1.1     gridExtra_2.3.1
## [107] xfun_0.61                   graphlayouts_1.2.5
## [109] stringi_1.8.9               lazyeval_0.2.3
## [111] ggfun_0.2.2                 yaml_2.3.12
```

```
## [113] ReactomePA_1.56.0          evaluate_1.0.5
## [115] codetools_0.2-20           ggraph_2.2.2
## [117] qvalue_2.44.0              gdtools_0.5.1
## [119] tibble_3.3.1               graph_1.90.0
## [121] ggplotify_0.1.4            cli_3.6.6
## [123] xtable_1.8-4               systemfonts_1.3.2
## [125] processx_3.9.0             dichromat_2.0-0.1
## [127] Rcpp_1.1.2                 png_0.1-9
## [129] XML_3.99-0.25              parallel_4.6.1
## [131] ggplot2_4.0.3              blob_1.3.0
## [133] clusterProfiler_4.20.2     DOSE_4.6.0
## [135] sparseMatrixStats_1.24.0   SpatialExperiment_1.22.0
## [137] GSEABase_1.74.0            viridisLite_0.4.3
## [139] tidytree_0.4.8             ggiraph_0.9.6
## [141] enrichit_0.2.6             scales_1.4.0
## [143] purrr_1.2.2                crayon_1.5.3
## [145] GetoptLong_1.1.1           rlang_1.3.0
## [147] KEGGREST_1.52.2
```